

Control-Flow Attestation on the S3K Separation Kernel

Giovanni Prete

March 21, 2026

Abstract & Introduction

This project implements a software-based prototype of Control-Flow Attestation (C-FLAT) on the S3K separation kernel, a capability-based operating system designed for embedded RISC-V devices. Our architecture achieves robust security by integrating:

1. Compile-time control-flow policy extraction and automatic basic block instrumentation via a custom LLVM compiler pass.
2. Kernel-level exception-based branch trapping and instruction decoding to securely log runtime control-flow events in a protected queue.
3. A privileged user-space monitor (running as PID 0) that consumes trace events, validates transitions against the extracted policy, enforces a shadow stack for call-return pairing, and performs loop folding to prevent path combinatorial explosion.

The resulting system provides a scale-invariant attestation fingerprint, enabling a remote verifier to verify that the application has executed exactly as intended without unauthorized deviations.

1 Countered Vulnerabilities

Implementing dynamic tracking mechanisms within an operating system kernel introduces security risks. If the logging, exception-handling, and syscall paths are not isolated, the attestation system itself can become a vector for exploitation. In this project, three critical vulnerabilities within the S3K kernel instrumentation and tracing interfaces were identified and patched.

1.1 Kernel CFA Event Buffer Overflow

The kernel intercepts control-flow branches and caches them in a central buffer before the monitor retrieves them.

- **Vulnerability Analysis:** The monitored application runs at native speed, while the monitor must query and verify events. If the application is malicious or generates high-frequency branch events, the kernel buffer can overflow. In a naive implementation, this would result in an unchecked wrap-around (overwriting trace records) or an out-of-bounds array write. In the context of a kernel, an out-of-bounds write can corrupt critical kernel structures (such as process tables, scheduler lists, or capability nodes), leading to kernel panic or local privilege escalation.
- **Countermeasure Implementation:** We implemented a robust circular ring buffer of size 1024 (`CFA_BUF_SIZE`) inside `kernel/src/cfa.c`. On pushing a new event, the kernel checks the buffer index state before executing any memory write:

```
int next_head = (cfa_head + 1) % CFA_BUF_SIZE;
if (next_head == cfa_tail) {
    cfa_overflow = 1; // Sticky overflow flag checked by monitor
    return;
}
```

When the head catches up to the tail, the buffer is marked as full, the write is dropped, and the sticky `cfa_overflow` flag is set. The user-space monitor verifies this flag upon each event consumption; if it is set, the attestation is immediately invalidated, preventing an attacker from executing undetected paths by blinding the monitor via buffer exhaustion.

1.2 Unauthorized Access to Control-Flow Trace Data

Because the execution path of a process contains details of its logical states, cryptographic operations, and private data offsets, protecting the trace database is critical.

- **Vulnerability Analysis:** If the system call to retrieve tracing events (`SYS_CFA_GET_EVENT`) is exposed to any process, an unprivileged process could execute it. A rogue application could spy on another process's control flow, allowing it to perform side-channel attacks to extract cryptographic keys or private algorithms. Furthermore, an attacker could maliciously poll and consume events from the queue, causing the monitor to miss transition records and fail the attestation.
- **Countermeasure Implementation:** S3K is a capability-based kernel enforcing strict spatial and process isolation. In `kernel/src/syscall.c`, we implemented a strict process check within the syscall validator:

```
proc_t *handle_cfa_get_event(proc_t *const p, const sys_args_t *args)
{
    if (p->pid != 0) { // PID 0 is reserved for the privileged monitor
        p->regs[REG_T0] = ERR_INVALID_SYSCALL;
        return p;
    }
    // ... retrieval of trace data ...
}
```

Only the monitor process, running under Process ID 0, is permitted to read events. Syscalls from other PIDs are rejected with an `ERR_INVALID_SYSCALL` code, preserving trace confidentiality and attestation integrity.

1.3 Event Type/Role Evasion

To perform dynamic validation (such as matching calls and returns), the monitor must receive the type/role of the executing block.

- **Vulnerability Analysis:** If the application only transmits its basic block ID during execution, the monitor cannot determine the control-flow context. An attacker exploiting a memory corruption vulnerability could chain ROP gadgets together to execute a sequence of blocks. If the monitor cannot verify the semantic role of these blocks (e.g. distinguishing a function call from a normal jump), it might accept an illegal transition.
- **Countermeasure Implementation:** We updated the LLVM pass to record block roles and propagate them to the kernel. Before triggering the breakpoint, the application encodes the role in register `t1` (with the block ID in `t0`). The kernel exception handler decodes this register:

```
uint64_t bbid = proc->regs[REG_T0];
uint64_t event_type = proc->regs[REG_T1];
...
uint8_t is_call = (event_type == 1 || event_type == 3) ? 1 : 0;
uint8_t is_return = (event_type == 2) ? 1 : 0;
cfa_push_event(proc->regs[REG_PC], bbid, (uint8_t)event_type, is_call, is_return);
```

This ensures that structural properties of the branch (such as function entry or exit) are propagated securely to the monitor.

2 Design Choices

Developing a control-flow attestation system for embedded architectures requires balancing security and execution overhead. Below are our key design choices:

2.1 Separation of Mechanism and Policy (User-Space Verification)

Rather than evaluating transition graphs and loops within the kernel, the project uses a separate user-space process (PID 0) as the CFA Monitor. Performing graph traversal, policy lookup, and cryptographic mixing inside the kernel increases code size and complexity, violating the microkernel philosophy. The S3K kernel only handles the mechanism (trapping events and writing them to the circular queue), while the user-space monitor enforces the policy.

2.2 Compiler-Based Static Analysis & Automatic Instrumentation

The control-flow policy is extracted statically at compile time using a custom LLVM Pass (`CFAGeneratorPass`). Program source code is compiled into LLVM Intermediate Representation (IR). The LLVM Pass automatically splits basic blocks after call instructions, assigns stable basic block IDs, and inserts call probes to `cfa_bb_enter(bbid, flags)` at the beginning of every block. This eliminates the need for manual binary rewriting, which is architecture-specific, and hard to maintain.

2.3 Exception-Based Software Trapping

To intercept basic block execution on RISC-V, the compiler pass inserts the inline assembly instruction `ebreak` inside the probe function. When the application hits an `ebreak`, it triggers a breakpoint exception (`mcause == 3`) handled by the S3K kernel. This hardware-isolated context switch allows the kernel to log the execution event securely. While hardware branch-tracing extensions would offer lower overhead, they are not universally available on low-end embedded microcontrollers. The exception-based software trapping provides a portable emulation of hardware tracing.

2.4 Loop Path Folding

A major challenge in control-flow attestation is the combinatorial explosion of execution paths caused by loops. A loop that runs N times with multiple paths inside it will generate 2^N possible hash chains, making verification impossible for the verifier. The monitor implements loop path folding. The LLVM pass extracts the structure of each loop (depth, header block, body edges, and exit edges) into the policy. At runtime, when the monitor detects that an edge belongs to a loop body, it suspends the main running hash calculation. It records the path taken within that single iteration and hashes it. If the loop repeats the same iteration path, the monitor simply increments an iteration counter in a path bucket. When the loop exits, the monitor mixes the loop ID, the paths taken, and their iteration counts into a single loop digest:

$$\text{Digest} = H(\text{LoopID} \parallel \text{Path}_1 \parallel \text{Count}_1 \parallel \dots \parallel \text{Path}_k \parallel \text{Count}_k)$$

This digest is then mixed back into the main `running_hash`. This restricts the size of the attestation response and keeps verification deterministic, regardless of the loop iteration count.

2.5 Shadow Stack for Call-Return Matching

Static CFG representation is context-insensitive, meaning it does not capture the call-history. In a static CFG, a function return instruction is allowed to branch to any of the function's call sites. This permits "calling-context attacks" where an attacker calls a function from site A but returns to site B . To enforce context sensitivity, the monitor maintains a dedicated shadow stack. When a block of type `Call` is executed, the monitor calculates its static return site using `policy_call_returns` and pushes it onto the shadow stack. When a `Return` block is executed, the monitor pops the expected return site from the shadow stack and verifies it against the actual destination. If it does not match, execution is halted.

3 Implementation Details

The prototype spans three main components: the LLVM pass, the S3K kernel exception handler, and the monitor verification engine.

3.1 LLVM Pass and Policy Serialization

The pass compiles as a dynamic library (`CFAGeneratorPass.dylib`) and executes during the optimization phase of `opt`.

1. Block Splitting: It iterates through the module's functions and identifies all call sites. To ensure call and return-site transitions are explicit, it splits the basic block immediately after the call instruction:

```
BasicBlock *OldBB = CB->getParent();
OldBB->splitBasicBlock(CB->getNextNode(), "call.ret");
```

2. Stable ID Assignment: It assigns a monotonic ID to each basic block. These IDs are used as the block addresses in the policy.

3. Instrumentation Insertion: It generates call instructions to `cfa_bb_enter` at the beginning of each block, passing the block ID and flags.
4. Metadata Serialization: It outputs `cfa_policy.json`, representing the static CFG:
 - `edges`: transitions between blocks.
 - `loops`: body and exit edges of loops.
 - `bb_types`: basic block classifications.
 - `call_returns`: matching pairs of call blocks and return targets.
 - `indirect_targets`: possible entry blocks of address-taken functions.

3.2 Kernel Exception Handler and Instruction Decoder

In `kernel/src/exception.c`, the breakpoint exception handler processes the traps generated by `cfa_bb_enter`. The handler extracts the instruction at the current PC. If it is a compressed instruction (16-bit), it sets `ebreak_len` to 2; otherwise, it is 4. It advances the process's saved PC by `ebreak_len` to resume execution past the probe. For branch-probe testing (where we trace branches dynamically in QEMU), the kernel implements a branch target decoder (`_decode_branch_target`):

- **JAL (0x6f)**: Extracts the 20-bit sign-extended immediate offset and adds it to the PC.
- **JALR (0x67)**: Extracts the source register, reads its saved value from the process state, adds the immediate, and masks the lowest bit.
- **Conditional Branch (0x63)**: Decodes the condition (`funct3`), reads the registers `rs1` and `rs2`, evaluates the branch condition, and sets the target PC to the target offset if the condition is met, or PC+4 if not.

3.3 Monitor Engine and Hash Mixing

The monitor (`monitor/main.c`) runs in an infinite loop, pulling events via the syscall. It maintains: An array `shadow_stack[64]` tracked by a stack pointer `shadow_sp`. An array of `loop_ctx_t` tracking the execution of active loops:

```
typedef struct {
    int active;
    uint64_t stage_hash;
    uint64_t path_hash;
    int path_len;
    loop_path_bucket_t buckets[MAX_LOOP_PATHS];
    int num_buckets;
} loop_ctx_t;
```

Digest Mixing: The mixing function uses a shift-rotate-xor mixing step to incorporate transitions:

```
static uint64_t cfa_mix(uint64_t state, uint64_t value) {
    return (state << 5) ^ (state >> 59) ^ value;
}
```

Strict Rejection: In enforcement mode (`CFA_STRICT_ENFORCEMENT=1`), a violation calls `s3k_mon_suspend` on the monitored process (APP1) and enters an infinite loop, freezing the process and signaling attestation failure.

4 Why Software Counters the Selected Attacks

Control-flow attestation protects the embedded application against a wide range of runtime attacks by enforcing strict path verification.

4.1 Code Injection

In a code injection attack, an attacker exploits a buffer overflow to inject shellcode into data memory (stack or heap) and redirects execution to it. Our platform enforces Data Execution Prevention, preventing execution from writable pages. If the attacker bypasses DEP (e.g., via multi-stage exploits), any execution of injected code will run instructions outside the benign binary. These instructions will not execute the instrumentation probes (`cfa_bb_enter`) or will execute invalid probes. The monitor will detect a missing probe sequence or invalid block transition, suspending the application immediately.

4.2 Return-Oriented Programming (ROP)

ROP attacks hijack the stack pointer to chain existing code snippets ("gadgets") ending in return instructions.

If a gadget jumps to the middle of a basic block, the execution bypasses the `cfa_bb_enter` probe at the start of that block. The monitor will detect that the execution skipped a block or did not generate the expected event, causing a mismatch in the transition tracking. If the gadget executes an entire basic block but jumps to an invalid destination, the monitor detects that the target address is not in the static CFG edge list. If the attacker chains functions (e.g. executing a return that jumps to a valid function entry site), the monitor's shadow stack detects the mismatch. Since the function was not called via a valid `Call` instruction, the return address popped from the shadow stack will not match the target, triggering a violation.

4.3 Jump-Oriented Programming (JOP)

JOP attacks use indirect jumps (`jalr` in RISC-V) to jump through gadgets. The LLVM pass identifies all valid targets of indirect branches (address-taken functions or labels) and outputs them in `policy_indirect_targets`. The monitor verifies every indirect transition against this list. If the JOP chain attempts to jump to an arbitrary instruction that is not in the target list, the monitor rejects the transition. Furthermore, the LLVM pass identifies inline assembly branches (like `j target_3` in `jop/main.c`) and flags them as `INLINE_ASM_BRANCH` (type 5). Because inline assembly branches are opaque and cannot be statically analyzed, the monitor treats them as invalid, preventing attackers from abusing embedded asm blocks to bypass CFG rules.

4.4 Function Reuse

These attacks chain legitimate virtual functions. Since C-FLAT attests the exact block order and loop counts, the out-of-order execution sequence yields an unrecognized path hash, failing attestation.

4.5 Non-Control-Data Attacks

These attacks corrupt decision variables (like authentication flags or loop bounds) to execute a valid but unauthorized path.

If the attacker forces the execution of a privileged branch instead of an unprivileged branch (e.g. changing `auth = true`), the application executes a different sequence of basic blocks. This alters the cumulative `running_hash` value. When the verifier checks the final hash, it will not match the expected hash for the unprivileged operation, revealing the unauthorized path execution. If the attacker corrupts a loop count variable, the loop executes M times instead of N . The monitor's loop folding records the loop path with an iteration count of M in the path bucket. The resulting loop digest mixed into `running_hash` will be different, allowing the verifier to detect the alteration.

5 Individual Contribution: Giovanni Prete

I implemented the core validation, exception decoding, and scripting automation:

5.1 LLVM Pass Development

I developed the LLVM compiler pass (`CFGGeneratorPass.cpp`). I implemented the split logic (`splitBasicBlock`) that handles basic block subdivision after call instructions to ensure call-return transitions could be verified with context sensitivity. I also mapped basic blocks to numerical identifiers (types 0 to 5) depending on their terminating instructions (direct/indirect calls, returns, switch statements, and inline assembly) and wrote the serialization logic to export the policy.

5.2 Kernel Trapping & Decode Logic

I modified the kernel exception trap handler in `kernel/src/exception.c` to process the traps generated by `cfa_bb_enter`. I designed the breakpoint trapping logic for RISC-V, including the instruction parser that determines whether the trapped breakpoint instruction is compressed (2-bit `ebreak`) or standard (4-bit `ebreak`). I also developed the branch target instruction decoder (`_decode_branch_target`) in C, which parses RISC-V opcodes (`0x6f` for JAL, `0x67` for JALR, and `0x63` for conditional branches) and computes target addresses dynamically.

5.3 Kernel Security Enhancements

I resolved two critical security vulnerabilities in the kernel:

- Implemented the circular event log ring buffer with sticky overflow detection in `cfa.c`.
- Restricted event retrieval to Process 0 in `syscall.c` to ensure process boundary integrity.

5.4 Monitor Verification & Loop Folding

I wrote the monitor verification engine in `monitor/main.c`. I implemented the shadow stack push/pop mechanism that matches return instructions against their callers, enforcing context-sensitive control-flow integrity. I also designed the loop folding state machine and created the `loop_ctx_t` structure. When a loop edge is detected, the monitor suspends the main hash chain, hashes the iteration path, registers it in a `loop_path_bucket_t` bucket, and increments the execution count. Upon loop exit, the monitor computes a final loop digest and mixes it into `running_hash`.

5.5 Pipeline Automation

I wrote the automation scripts to enable the profiling and enforcement pipeline:

- `merge_cfg.py`: A Python script that parses the monitor's execution log in profiler mode, extracts dynamic edges logged with the `[DYNAMIC_EDGE_DISCOVERED]` prefix, merges them into `cfa_policy.json`, and preserves existing edge IDs.
- `policy_to_consts.py`: A helper script that parses `cfa_policy.json` and outputs `monitor/policy.h`, compiling the CFG directly into the monitor binary.
- `test.sh`: A shell script that automates the profiling pipeline.